

Caching filesystem access

Standa Lázníčka

Principal Software Engineer at Microsoft

Caching

- everybody knows what this is
 - accessing stuff is slow? Add $O(1)$ access buffer layer!



My service's
dependency
is too slow



Add a cache
in between!

Caching issues

- no infinite memory
- understand the usecase
- understand the constraints
- pick the right pattern
- investigate when caching stays efficient
- cache record invalidation

Caching patterns

- [by datastore access](#)
 - cache-aside, read-through, write-through, write-behind, refresh-ahead
- [by size-limitation caching strategy](#)
 - least recently used, least frequently used, FIFO, LIFO, random replacement
- LRU reference implementations
 - github.com/hashicorp/golang-lru
 - MPL licence [may not fit everyone](#)
 - k8s.io/Utils/INTERNAL/third_party/forked/golang/golang-lru/lru.go
 - a bit more dated

Kubelet image access authorization

- guard access to private images pulled onto the node
- **usecase**
 - authorization records = JSON on disk
 - concurrency
- **constraints**
 - resources shared with orchestrated processes
 - recommended [max 110*N processes](#), N is a small number
 - some may run it on edge
- Kubernetes [KEP 2535](#)

Dealing with memory constraints

k8s LRU

```
type Cache struct {  
    // MaxEntries is the maximum number of cache entries before  
    // an item is evicted. Zero means no limit.  
    MaxEntries int  
  
    // OnEvicted optionally specifies a callback function to be  
    // executed when an entry is purged from the cache.  
    OnEvicted func(key Key, value interface{})  
  
    ll    *list.List  
    cache map[interface{}]*list.Element  
}
```

- freshly-read and new items on top of ll
- write on full cache
 - last ll item removed
 - OnEvicted() called for the removed elem

Dealing with memory constraints - real life

- combining access
 - LRU cache
 - data storage on misses
- LRU cache misses expensive
- **cache authority** (with a warm cache)
 - $size(datastore) < size(cache)$
 - size = mandatory parameter
 - fine-tuning effectivity
- strive for authoritative mode

Concurrent access

- protect access to datastore
- avoiding lock contention
 - use named locks - `datastruct.Lock(name)`
- `sync.Map[sync.Mutex]`
 - memory constraints still apply!
 - when is it safe to free a lock from the map?
- limited prealloc'd lock set?

Concurrent access - striped lock set

```
type StripedLockSet struct {
    locks []sync.Mutex
    size  int32
}

func (s *StripedLockSet) Lock(key string) {
    s.locks[keyToID(key, s.size)].Lock()
}

func (s *StripedLockSet) GlobalLock() {
    for i := range s.locks {
        s.locks[i].Lock()
    }
}

func keyToID(key string, sliceSize int32) uint32 {
    h := fnv.New32()
    h.Write([]byte(key))
    return h.Sum32() % uint32(sliceSize)
}
```

Summarized - Write(record)

1. `cache.locks.Lock(record.Name); defer cache.locks.Unlock(record.Name)`
2. `datastore.Write(record)`
3. `cache.lru.Set(record.Name, record)`

Summarized - Read(recordName)

1. cache.lru.Get(recordName) # cheap lookup
 - *miss!*
2. cache.locks.Lock(recordName); defer cache.locks.Unlock(recordName)
3. cache.lru.Get(recordName) # could've been written
 - *miss!*
4. cache.isAuthoritative # isAuthoritative is atomic.Bool
 - *false!*
5. datastore.Get(recordName)
 - *hit!*
6. cache.lru.Set(recordName, record) # might cause isAuthoritative=false

Benchmarking

- caching ✕ direct access
- parameters?
 - authoritative ✕ non-authoritative
 - non-authoritative: a little ✕ a lot
 - cache hits ✕ cache misses
 - concurrency? Lock contention!

Lock contention benchmarks?

- how to test?
 - no good tools to do "LockSet size = 2, run 4 goroutines in loop"
 - `b.SetParallelism(p)` runs $p * GOMAXPROCS$ test goroutines
- set *StripedLockSet* size to `runtime.NumCPU()`
- p
 - `== 1` -> true parallelism
 - `>1` -> concurrency

Benchmark structure

```
func BenchmarkStuff(b *testing.B) {  
    // initialize structures, prepare test data  
    for b.Loop() {  
        // measured code  
    }  
    // cleanup  
}  
  
func BenchmarkStuffInParallel(b *testing.B) {  
    // initialize structures, prepare test data  
    b.SetParallelism(<int greater than 0>)  
    b.ResetTimer()  
    b.RunParallel(func(pb *testing.PB) {  
        for pb.Next() {  
            // measured code  
        }  
    })  
    b.StopTimer()  
    // cleanup  
  
}
```

Running benchmarks

- isolated, stable machine with little to no activity
- test minimal code units
- run each benchmark multiple times
 - analyze deviations
- `go test benchmem -count 10 -timeout 40m -run ^$ -bench ^BMName$ '
<path>' > result.txt`
- golang.org/x/perf/cmd/benchstat
 - `benchstat -col /Type -filter '/Type:(FileSystemAccessor OR
InMemoryCache-Authoritative)' result.txt`

Example results

[./mustattemptpull_hit.txt](#)

[./mustattemptpull_hit_benchstat.txt](#)

Read more about it!

- KEP: <https://kep.k8s.io/2535>
- Cache implementation: [kubernetes/kubernetes/pull/131882](https://github.com/kubernetes/kubernetes/pull/131882)
- Benchmark implementation and results - [kubernetes/kubernetes/pull/131864](https://github.com/kubernetes/kubernetes/pull/131864)

Thank you

Standa Lázníčka

Principal Software Engineer at Microsoft

stanislav.laznicka@microsoft.com

<https://github.com/stlaz>

<https://www.linkedin.com/in/slaznicka/>

stlaz on k8s Slack